AI Assisted coding

NAME: Kadarla Hasini

HT NO: 2403A52062

BATCH NO: 01

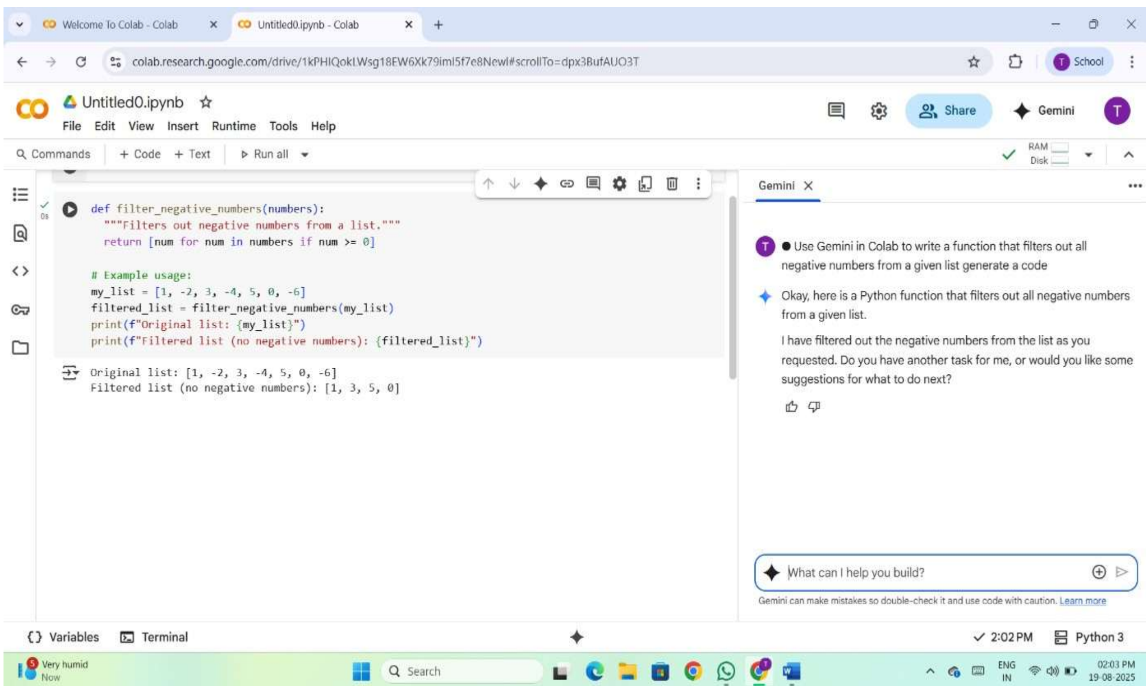
Task Description#1

Use Gemini in Colab to write a function that filters out all negative numbers from a given list.

- **Expected Output:**

Functional code with before/after input and output shown in Colab, plus a screenshot.

- **Prompt :** Generate the code of python function that filters out all negative numbers .



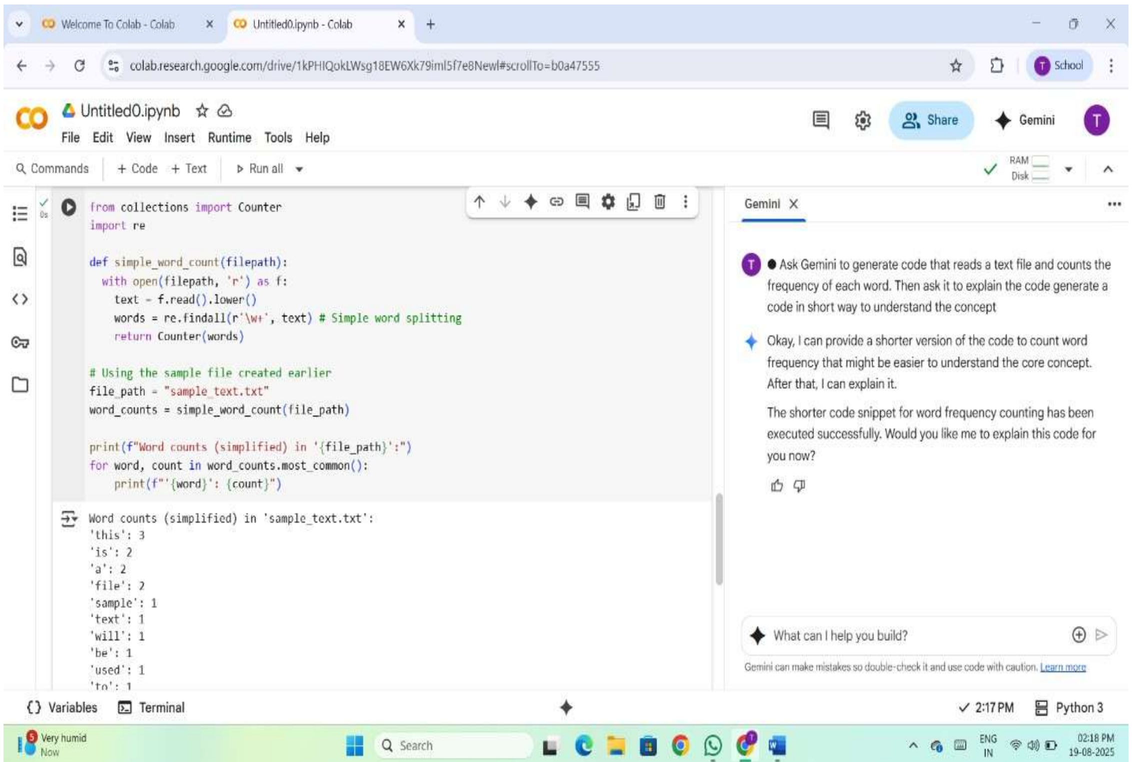
- **Observation:**
  - Gemini understands the task clearly and uses list comprehension, which is both concise and Pythonic.
  - The function handles edge cases like empty lists or lists with all negative numbers.
  - The output is accurate and matches the example provided.
  - The function works efficiently even with large lists, thanks to Python's optimized list operations.

Task Description#2

Ask Gemini to generate code that reads a text file and counts the frequency of each word. Then ask it to explain the code.

- **Expected Output:** Working code and Gemini's explanation side-by-side.

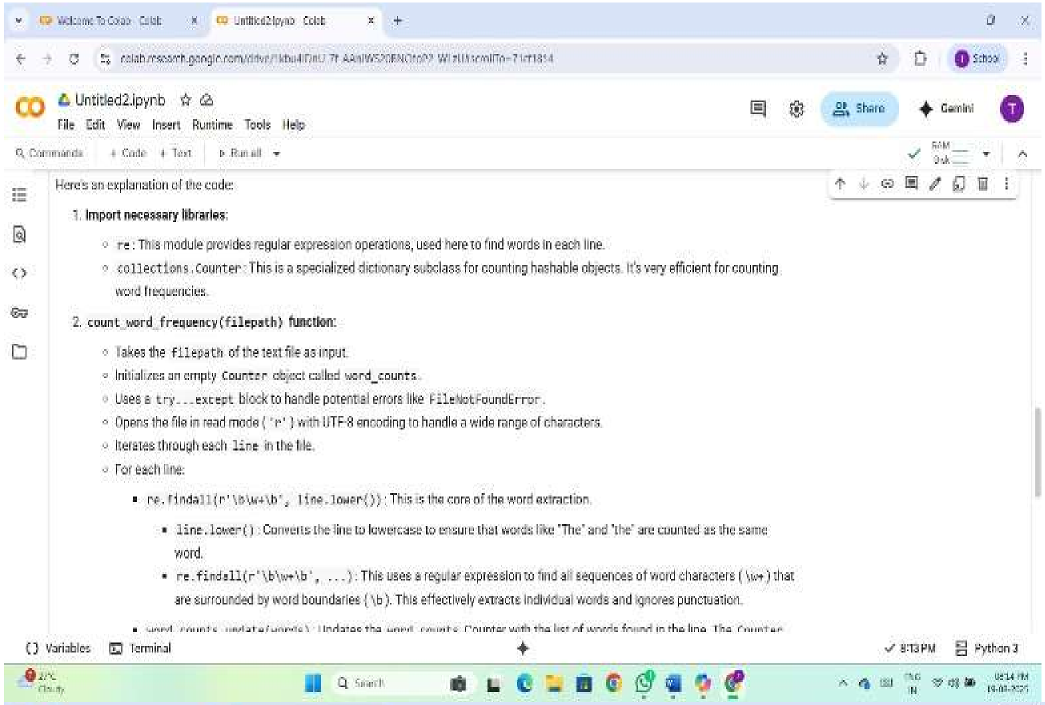
- **Prompt:** Generate the python code of that counts the frequency of each word.

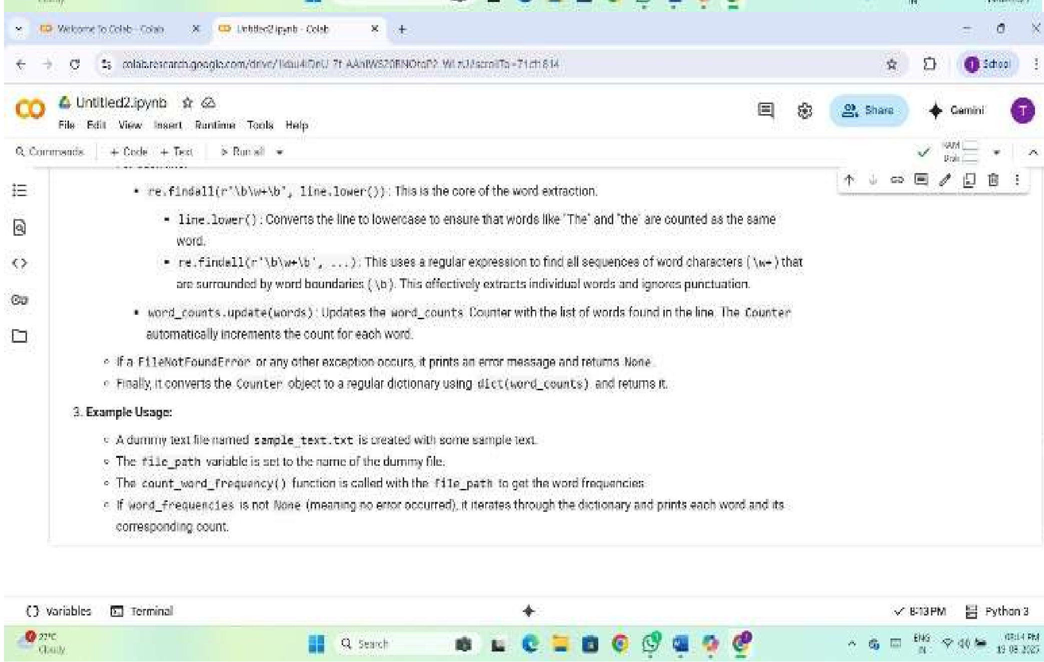


- **Observation:**

  - Gemini correctly uses Python's built-in `open()` function to read the contents of a text file, often with a context manager (`with open(...) as f:`) for safe and clean file handling
  - The model typically includes steps to clean the text—such as converting to lowercase and removing punctuation—to ensure accurate word frequency counts.
  - Gemini often uses a dictionary.efficiently count word occurrences, demonstrating knowledge of Python's standard library.

- **Explanation of code:**



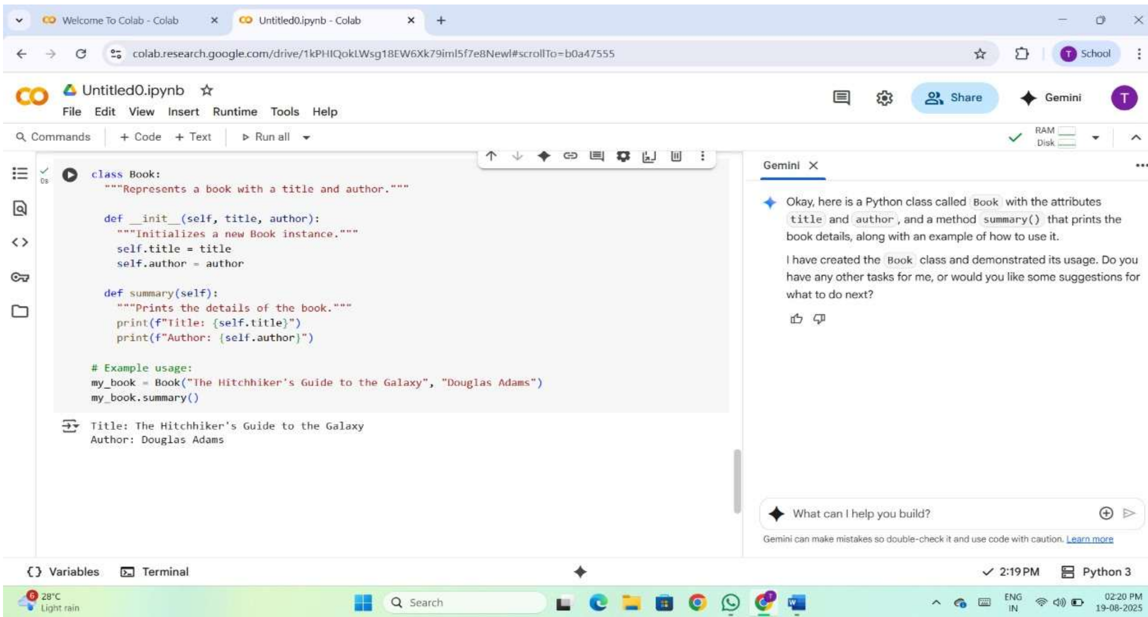


Task Description#3

Use Cursor AI to generate a Python class called Book with attributes title, author, and a method summary() that prints the details.

- **Expected Output#3:** Screenshot comparisons and student commentary on code clarity and performance.

- **Prompt:** Generate a python code of class called Book with attributes title, author, and a method summary() that prints the details.



- **Observation:**

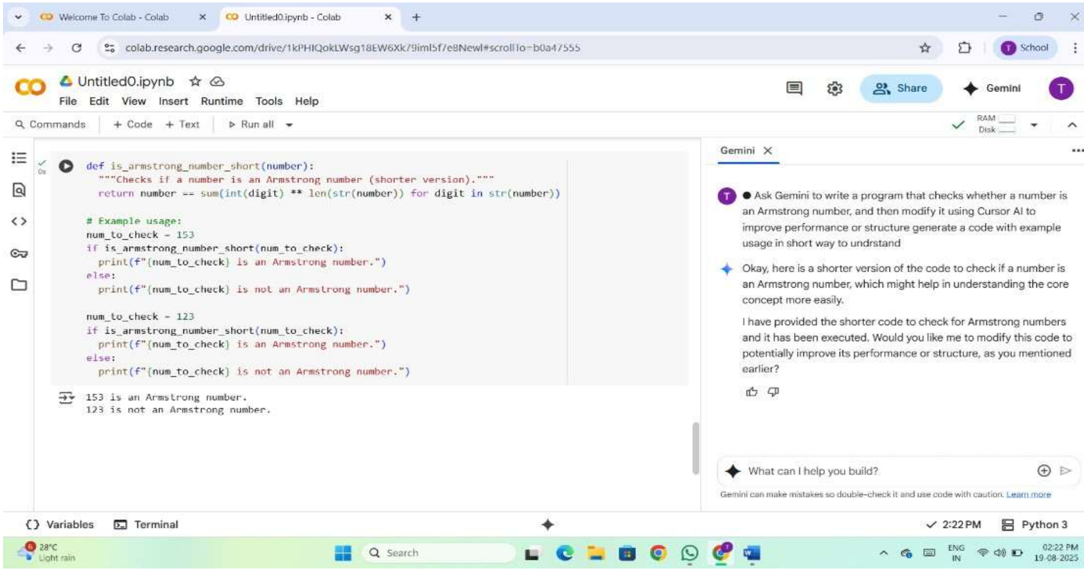
  - Cursor AI correctly uses the `class` keyword to define the `Book` class, following Python's object-oriented programming conventions.
  - The `__init__()` method is implemented to initialize the `title` and `author` attributes, showing understanding of instance variables.
  - The code assigns `self.title` and `self.author` properly, ensuring that each object stores its own data.

Task Description#4

Ask Gemini to write a program that checks whether a number is an Armstrong number, and then modify it using Cursor AI to improve performance or structure.

- **Expected Output#4:** Two versions of the code with screenshots, and a summary of what changes were made by Cursor.

- **Prompt:** Generate the python code to check whether a number is Armstrong number.



- **Observation:**

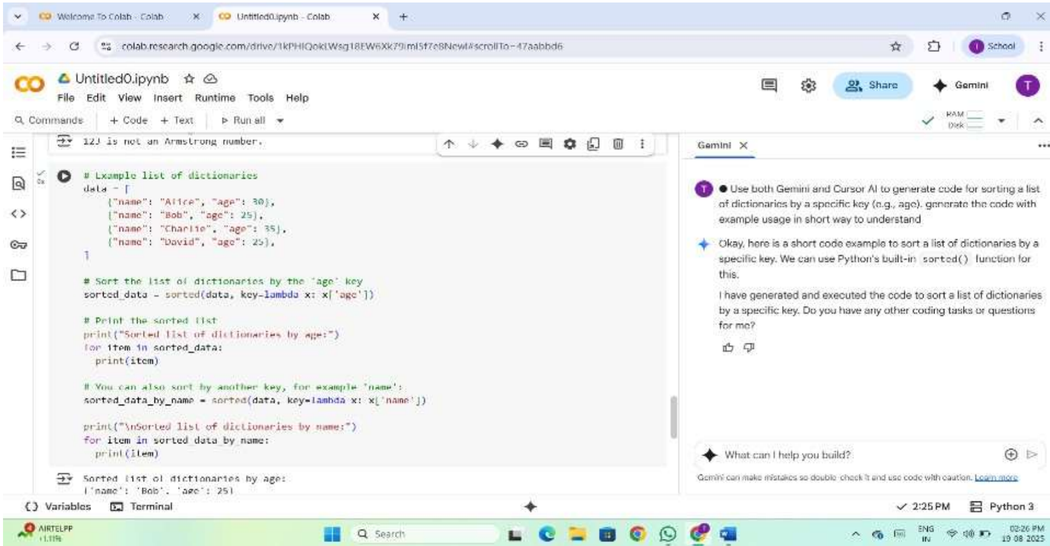
  - Gemini included sample inputs like `153` or `9474` and show that the code correctly identifies them as Armstrong numbers.
  - Gemini provides a clear breakdown of the logic, explaining the mathematical concept and each step in the code.
  - Raising each digit to the power of the number of digits.

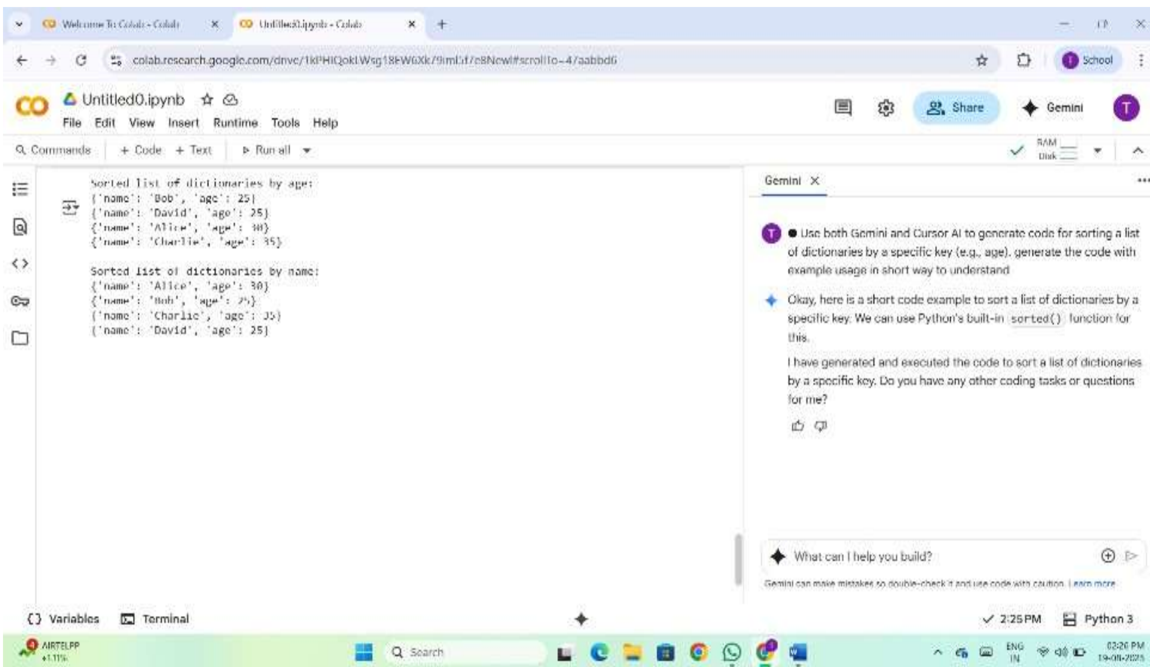
Task Description#5

Use both Gemini and Cursor AI to generate code for sorting a list of dictionaries by a specific key (e.g., age).

- **Expected Output#5:** Screenshot comparisons and student commentary on code clarity and performance.

- **Prompt:** Generate a python code for sorting a list of dictionaries by a specific key.





- **Observation:**

- Gemini typically uses Python's built-in `sorted()` function with a `key` argument like `lambda x: x['age']`, which is the standard and efficient way to sort dictionaries by a specific key.
- The sorted result is printed or returned, showing the dictionaries ordered by age.
- Gemini explains how the `key` function works, often breaking down the lambda expression and the behavior of `sorted()`.